

Chapter 08

Document Loaders

The first building block of any RAG pipeline — how LangChain pulls data from any source and standardizes it into Document objects.

WHAT THIS CHAPTER COVERS

- ▶ What RAG is and why it matters
- ▶ The four core RAG components
- ▶ The Document object standard
- ▶ TextLoader for plain text files
- ▶ PyPDFLoader for PDF documents
- ▶ DirectoryLoader for bulk loading
- ▶ load() vs lazy_load() semantics
- ▶ WebBaseLoader & CSVLoader

Document Loaders

RECAP — WHERE WE LEFT OFF

The previous chapters covered LangChain's core components (Models, Prompts, Chains) and core concepts (Runnables, LCEL). With the fundamentals in place, this chapter begins a new arc: building **RAG (Retrieval-Augmented Generation) based applications** using LangChain, starting with the very first component in that pipeline — Document Loaders.

1. What Is RAG?

Definition

RAG (Retrieval-Augmented Generation) is a technique that combines information retrieval with language generation. A model retrieves relevant documents from an external knowledge base and then uses them as context to generate accurate, grounded responses.

Why It Exists

A standard LLM like ChatGPT is limited in situations where it has no access to certain data:

- **Current affairs** — the model is trained on past data and may not know about recent, day-to-day events.
- **Personal data** — questions about a user's private emails or documents that the model has never seen.
- **Company/internal documentation** — proprietary knowledge that was never part of the model's training data.

In all these cases, the LLM simply does not have the information required to answer — RAG solves this by connecting the LLM to an *external knowledge base* (a company database, a set of PDFs, personal documents, etc.). When a user asks a question the LLM cannot answer directly, it looks up the answer in that knowledge base and uses it to generate a response.

Benefits of RAG

Benefit	Explanation
Up-to-date information	The knowledge base can be refreshed independently of the LLM's training cycle.
Privacy	Sensitive documents never need to be uploaded to a third-party chat interface — they stay in your own knowledge base.
No document size limit	Large documents are split into chunks and processed piece by piece, bypassing the LLM's context-length limits.

REAL WORLD INSIGHT

RAG is one of the most powerful trends in the GenAI industry precisely because it removes the two biggest limitations of a plain LLM: stale knowledge and inability to reason over private data.

🔄 Quick Revision — What Is RAG?

- RAG combines information retrieval (from an external knowledge base) with language generation (from an LLM).
- Plain LLMs struggle with current affairs, personal data, and private/company documentation.
- RAG connects an LLM to an external knowledge base so it can look up answers it doesn't already know.
- Key benefits: up-to-date answers, better privacy, and no practical document size limit.

2. The Core Components of a RAG Application

Definition

Regardless of how complex a RAG architecture looks, almost every RAG-based application is built from the same four core components.

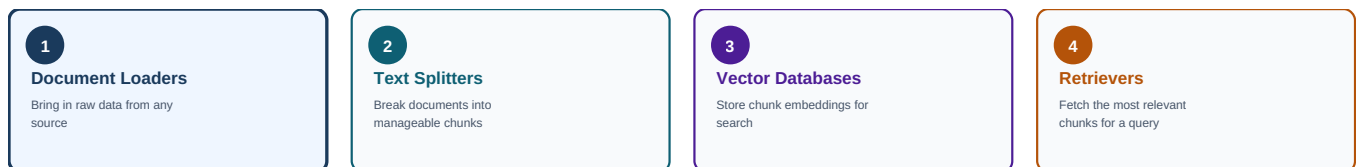


Figure 8.1 — The four core components used to build any RAG application.

1. DOCUMENT LOADERS

Bring raw data into LangChain from any source (files, databases, cloud storage, websites).

2. TEXT SPLITTERS

Break large documents into smaller, manageable chunks for embedding and retrieval.

3. VECTOR DATABASES

Store the embeddings of those chunks so they can be searched semantically.

4. RETRIEVERS

Fetch the most relevant chunks for a given user query at request time.

This chapter focuses entirely on the first component: **Document Loaders**. The remaining three components (Text Splitters, Vector Databases, Retrievers) are covered sequentially in the following chapters, after which the full RAG pipeline is assembled end-to-end.

🔄 Quick Revision — RAG Components

- Any RAG application is built primarily from four components: Document Loaders, Text Splitters, Vector Databases, and Retrievers.
- Document Loaders are the entry point — they bring raw data into LangChain.
- This chapter covers Document Loaders in depth; the other three components follow in later chapters.

3. What Are Document Loaders?

Definition

Document Loaders are components in LangChain used to load data from various sources into a standardized format, usually as **Document objects**, which can then be used for chunking, embedding, retrieval, and generation.

Why It Exists

Data required for RAG applications can come from hundreds of different sources — PDFs, plain text files, databases, cloud storage providers, websites, and more. To make this data usable by the rest of the LangChain ecosystem, it must be brought into a single, common format regardless of where it came from. Document Loaders solve this by standardizing every source into the same object type: a **Document**.

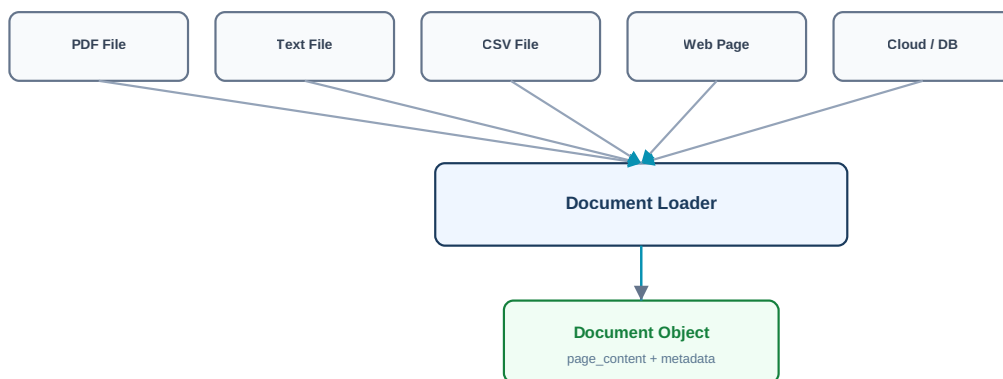


Figure 8.2 — Multiple raw data sources are standardized into a single Document object format.

How It Works — The Document Object

Every Document object produced by a loader has exactly two parts:

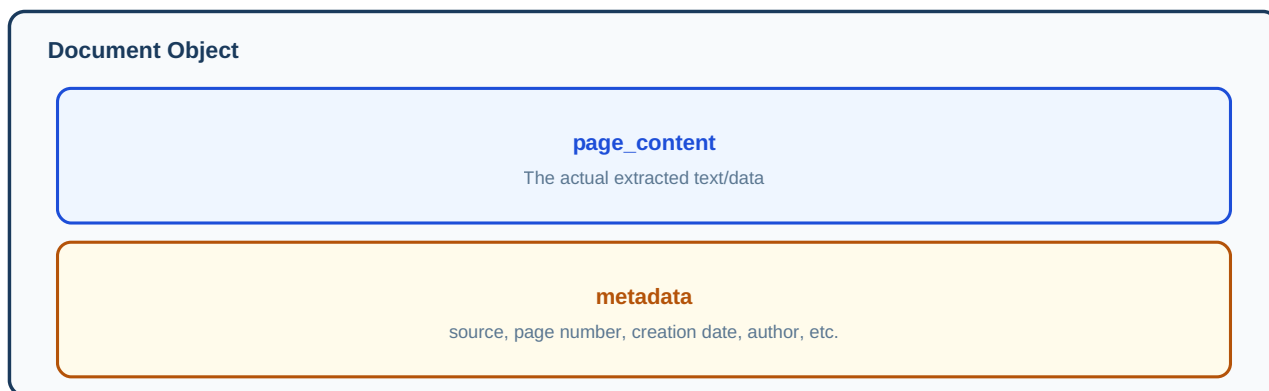


Figure 8.3 — Anatomy of a single Document object.

- **page_content** — the actual extracted content/data.
- **metadata** — supporting information such as source, creation date, last-modified date, author name, page number, etc.

IMPORTANT

Every document loader in LangChain, regardless of the source it reads from, always returns a **list of Document objects** — never a single raw object. Internally, LangChain divides the loaded content into multiple parts/pages and stores each part as one Document inside that list.

🔄 Quick Revision — Document Loaders

- Document Loaders standardize data from any source into Document objects.
- All document loaders live in the `langchain_community.document_loaders` package.
- A Document object has exactly two parts: `page_content` and `metadata`.
- Calling `.load()` on any document loader always returns a list of Document objects.

4. TextLoader

Definition

TextLoader is the simplest document loader in LangChain. Its only job is to pick up a plain text file and bring it into LangChain as a Document object.

When To Use It

Ideal for processing log files, code snippets, or transcripts (for example, a YouTube video transcript saved as a `.txt` file).

Example

```
from langchain_community.document_loaders import TextLoader
from langchain_groq import ChatGroq
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()

model = ChatGroq(model="llama-3.3-70b-versatile")

# Load a plain text file as a Document object
loader = TextLoader("cricket.txt", encoding="utf-8")

docs = loader.load() # returns a list of Document objects

parser = StrOutputParser()

prompt = PromptTemplate(
    template="Read the following poem : \n {poem} \n and generate a summary for it.",
    input_variables=["poem"]
)

chain = prompt | model | parser

# docs[0].page_content holds the raw extracted text of the file
print(chain.invoke({"poem": docs[0].page_content}))
```

Walking through the important lines:

- `TextLoader("cricket.txt", encoding="utf-8")` creates the loader object. The `encoding` parameter is usually needed only when the file contains special characters.
- `loader.load()` reads the file and returns a **list of Document objects** — for a single text file, this list contains exactly one Document.
- `docs[0].page_content` extracts the full text content of the file; `docs[0].metadata` extracts its metadata (in this case, just the source path).
- The extracted `page_content` is fed straight into a prompt (`{poem}`) as part of an LCEL chain (`prompt | model | parser`) that asks the LLM to summarize the poem.

BEST PRACTICE

The output of `loader.load()` can be piped directly into an LCEL chain (or wrapped in a `RunnableLambda`) so document loading becomes just another step in a `Runnable` pipeline.

Quick Revision — TextLoader

- TextLoader is the simplest LangChain document loader — it loads a plain .txt file into a Document object.
- Best suited for log files, code snippets, and transcripts.
- `encoding="utf-8"` is commonly needed when the file has special characters.
- A single text file produces a list with exactly one Document object.

5. PyPDFLoader

Definition

PyPDFLoader is the most commonly used PDF document loader in LangChain. It reads a PDF file and converts it into Document objects, internally relying on the **pypdf** library.

How It Works

PyPDFLoader works on a **page-by-page basis**. If a PDF has 23 pages, loading it produces exactly 23 Document objects — one per page — each with its own `page_content` and its own `metadata` (which includes the page number, source, total pages, and more).

Example

```
from langchain_community.document_loaders import PyPDFLoader

# Point the loader at the target PDF file
loader = PyPDFLoader("Interview_notes.pdf")

docs = loader.load() # one Document object per page of the PDF

print(docs[0].page_content) # text content of page 1
print(docs[0].metadata)     # page number, source, total pages, etc.
```

- `PyPDFLoader("Interview_notes.pdf")` creates the loader, pointed at the target PDF file.
- `loader.load()` returns a list of Document objects — one per page of the PDF.
- `docs[0].page_content` and `docs[0].metadata` give the text and metadata of the first page respectively.

COMMON MISTAKE

PyPDFLoader must have `pip install pypdf` installed on the machine — without it, the loader will fail. It also does not work well on scanned images or PDFs with complex, non-textual layouts.

Other PDF Loaders

PyPDFLoader is not the only PDF loader available in LangChain. Depending on the structure of the PDF, a different loader may work better:

Loader	Best For
PyPDFLoader	Simple, text-heavy PDFs
PDFPlumberLoader	PDFs with tabular data / tables to extract
UnstructuredPDFLoader	Scanned image PDFs, complex structure extraction
AmazonTextractPDFLoader	Scanned image PDFs (via AWS Textract)
PyMuPDFLoader	PDFs with heavy layout/formatting

BEST PRACTICE

Do not try to memorize every PDF loader in LangChain. Learn the underlying concept once, and refer to the official LangChain document loader documentation on a project-by-project basis when a specific loader is needed.

🔄 Quick Revision — PyPDFLoader

- PyPDFLoader converts a PDF into one Document object per page, using pypdf internally.
- A 23-page PDF produces 23 Document objects, each with its own `page_content` and `metadata`.
- PyPDFLoader struggles with scanned images and complex layouts.
- Alternative loaders exist for tables (PDFPlumberLoader), scanned images (UnstructuredPDFLoader, AmazonTextractPDFLoader), and complex layouts (PyMuPDFLoader).

6. DirectoryLoader

Definition

DirectoryLoader is a document loader that helps load multiple documents from inside a single directory (folder) in one operation, rather than loading each file individually.

Why It Exists

If a folder contains multiple PDFs or text files that must all be loaded together into LangChain, calling a single-file loader repeatedly by hand is tedious. DirectoryLoader wraps another loader class and applies it across every matching file in a directory.

Example

```
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader

loader = DirectoryLoader(
    path="books",          # Directory to look for files
    glob="*.pdf",          # Pattern match; this fetches every PDF in the folder
    loader_cls=PyPDFLoader # Which loader class to apply to each matched file
)

# Eager version (loads everything into memory at once):
# docs = loader.load()
# for doc in docs:
#     print(doc.metadata)

# Lazy version (returns a generator, one Document at a time):
docs = loader.lazy_load()

for doc in docs:
    print(doc.metadata)
```

- `path="books"` — the directory to search inside.
- `glob="*.pdf"` — a pattern describing which files to pick up. `*.pdf` means "every PDF file directly inside this folder"; patterns like `**/*.txt` would recurse into every subfolder for text files.
- `loader_cls=PyPDFLoader` — the actual loader class `DirectoryLoader` should apply to each matched file.

For three PDFs with 326, 392, and 468 pages respectively, `loader.load()` returns **1,186 Document objects** in total (326 + 392 + 468) — one Document per page, across all matched files. `DirectoryLoader` is not limited to PDFs; it works with any document loader class passed to `loader_cls`.

Quick Revision — DirectoryLoader

- `DirectoryLoader` loads multiple documents from a folder in a single operation.
- `glob` defines the file-matching pattern (e.g. `*.pdf`, `**/*.txt`).
- `loader_cls` specifies which underlying loader class to apply to each matched file.
- The total number of Documents returned equals the sum of pages/rows across every matched file.

7. load() vs lazy_load()

Why It Exists

Loading many large files at once (as seen with `DirectoryLoader` across three books) has two real problems:

1. **Slow to start** — loading all documents before returning anything takes noticeably longer as the number/size of files grows.

2. **High memory usage** — every single Document object is held in RAM simultaneously, which does not scale to hundreds of large files.

Every document loader in LangChain exposes two functions to address this: `load()` and `lazy_load()`.

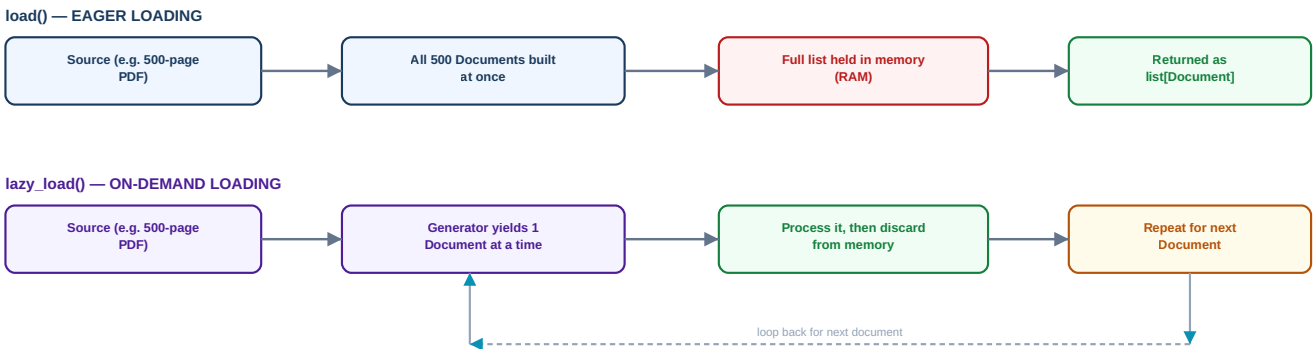


Figure 8.4 — `load()` eagerly builds every Document at once; `lazy_load()` yields one Document at a time.

Comparison

Aspect	<code>load()</code>	<code>lazy_load()</code>
Strategy	Eager loading	On-demand (lazy) loading
Return type	A list of Document objects	A generator of Document objects
Memory behavior	All documents loaded into memory at once	One document fetched into memory at a time, processed, then discarded
Best for	A small number of documents	A large number of documents / large files, or stream processing

BEST PRACTICE

Use `lazy_load()` whenever the number of documents is large or unpredictable. Because it returns a generator, you loop over it and process one Document object at a time instead of holding the entire collection in memory.

INTERVIEW TIP

If asked to explain the difference between `load()` and `lazy_load()`, lead with the return type (list vs. generator) and then connect it directly to the memory trade-off — that is the core of what interviewers are testing for.

Quick Revision — `load()` vs `lazy_load()`

- `load()` is eager: it loads every Document into memory at once and returns a list.
- `lazy_load()` is lazy: it returns a generator that yields one Document object at a time.
- `load()` is best for a small number of documents; `lazy_load()` is best for large numbers of documents or files.
- `lazy_load()` enables stream processing without high memory overhead.

8. WebBaseLoader

Definition

WebBaseLoader is a document loader used to load and extract text content directly from web pages, so that questions can be asked about that page's content.

How It Works

Internally, WebBaseLoader uses two Python libraries:

- **requests** — to make an HTTP request to the target web page.
- **BeautifulSoup** — to parse the page's HTML structure and extract clean, textual content from it.

Example

```
from langchain_community.document_loaders import WebBaseLoader
from langchain_groq import ChatGroq
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()

model = ChatGroq(model="llama-3.3-70b-versatile")
parser = StrOutputParser()

url = "https://www.amazon.in/gp/product/B0FPRCDS5J/..."

# requests fetches the HTML; BeautifulSoup strips it down to text
loader = WebBaseLoader(url)

docs = loader.load() # single URL -> single Document object

prompt = PromptTemplate(
    template="Extract the price and model of the given product page data : \n {page_data}",
    input_variables=["page_data"]
)

chain = prompt | model | parser

print(chain.invoke({"page_data": docs[0].page_content}))
```

- `WebBaseLoader(url)` creates the loader, pointed at a single URL (a list of URLs can also be passed — in that case, one Document is returned per URL).
- `loader.load()` fetches the page and strips out HTML tags, returning the visible text as a single Document.
- `docs[0].page_content` holds the extracted page text, which is then passed into a prompt asking the LLM to extract the price and model of the product from that page.

IMPORTANT

WebBaseLoader works best on **static** web pages (blogs, news articles, public informational pages) where the HTML already contains the content. For JavaScript-heavy pages that render content dynamically based on user interaction, SeleniumURLLoader is a better fit.

Quick Revision — WebBaseLoader

- WebBaseLoader loads and extracts text content from a web page URL.
- Internally combines requests (HTTP fetch) and BeautifulSoup (HTML parsing).
- A single URL returns a single Document object; a list of URLs returns one Document per URL.
- Works best on static pages; JavaScript-heavy pages need SeleniumURLLoader instead.

9. CSVLoader

Definition

CSVLoader is a document loader used to load CSV files into LangChain, typically so questions can be asked over the data using an LLM.

How It Works

CSVLoader creates **one Document object per row** of the CSV file. Each Document's `page_content` is a string listing every column name alongside its value for that row; the `metadata` includes the source file and the row index.

Example

```
from langchain_community.document_loaders import CSVLoader

# flights.csv has 5 columns and 400 rows of flight data
loader = CSVLoader("flights.csv")

docs = loader.load() # one Document object per row -> 400 Documents

print(len(docs))      # 400
print(docs[0].metadata) # {'source': 'flights.csv', 'row': 0}
print(docs[0].page_content) # "Flight_ID: F0001\nAirline: IndiGo\n..."
```

- `CSVLoader("flights.csv")` creates the loader, pointed at the target CSV file.
- `loader.load()` returns one Document per row — for a 400-row CSV, this returns 400 Document objects.
- `docs[0].metadata` and `docs[0].page_content` show the metadata (source, row number) and the column-name/value pairs for the first row respectively.

BEST PRACTICE

For very large CSV files, use `lazy_load()` instead of `load()` and loop through rows one at a time — this keeps memory usage low while still allowing row-level operations such as finding a maximum value in a particular column.

Quick Revision — CSVLoader

- CSVLoader converts each row of a CSV file into one Document object.
- `page_content` is a string of column-name/value pairs; metadata includes source and row index.
- A 400-row CSV produces 400 Document objects.
- `lazy_load()` is preferable for very large CSV files.

10. Other Document Loaders & Building a Custom Loader

Other Document Loaders

LangChain ships with hundreds of document loaders beyond the four covered in this chapter, organized into well-defined categories in the official documentation:

Web Pages

Cloud Storage (S3, Azure, Dropbox, Google Drive)

Social Platforms

Messaging Services

Productivity Tools (Git, Notion, etc.)

Common File Types (JSON, CSV, etc.)

YouTube Transcripts

It is neither necessary nor productive to study every available loader upfront. The recommended approach is to learn the four covered here thoroughly, then consult the official documentation for a specific loader only when a real project requires it.

Building a Custom Document Loader

When a data source has no existing LangChain loader, LangChain allows building a fully custom document loader. This is done by creating a class that inherits from LangChain's **BaseLoader** class, and then defining custom `load()` and/or `lazy_load()` logic inside it.

REAL WORLD INSIGHT

The entire library of document loaders that ship with LangChain today were built exactly this way — community programmers built custom loaders for their own use cases and contributed them back, which is why every document loader lives inside the `langchain_community` package.

Quick Revision — Other & Custom Loaders

- LangChain provides hundreds of document loaders across categories like web pages, cloud storage, and messaging platforms.
- Learn loaders on a project-by-project basis rather than studying the entire catalog upfront.
- A custom document loader is built by subclassing `BaseLoader` and implementing `load()/lazy_load()`.
- All existing community loaders were built using this same custom-loader mechanism.

K. Key Takeaways

- **RAG** combines information retrieval from an external knowledge base with LLM-based generation, solving problems around stale knowledge, private data, and document size limits.
- **Every RAG application** is built from four core components: Document Loaders, Text Splitters, Vector Databases, and Retrievers.
- **Document Loaders** standardize data from any source into Document objects, each made up of `page_content` and `metadata`.
- **TextLoader** loads plain text files; ideal for logs, code snippets, and transcripts.
- **PyPDFLoader** loads PDFs page-by-page, producing one Document per page; other PDF loaders exist for tables, scans, and complex layouts.
- **DirectoryLoader** applies a chosen loader class across every matching file inside a folder.
- **load() vs lazy_load()**: `load()` eagerly returns a full list of Documents; `lazy_load()` returns a generator that yields Documents one at a time, saving memory.
- **WebBaseLoader** extracts text content from static web pages using requests and BeautifulSoup.
- **CSVLoader** converts each row of a CSV file into one Document object.
- **Custom Document Loaders** can be built by subclassing `BaseLoader` when no existing loader fits a data source.

R. Revision Sheet

RAG

A technique combining retrieval from an external knowledge base with LLM generation to produce accurate, grounded answers.

Document Loader

A LangChain component that loads data from any source and standardizes it into a list of Document objects.

Document Object

The standard unit of data in LangChain, made of `page_content` (actual data) and metadata (source, page number, etc.).

TextLoader

Loads a plain text file into a single Document object.

PyPDFLoader

Loads a PDF page-by-page; each page becomes one Document object.

DirectoryLoader

Applies a given loader class across every file matching a glob pattern inside a folder.

load()

Eager loading — returns a full list of Document objects in memory at once.

lazy_load()

On-demand loading — returns a generator that yields one Document at a time.

WebBaseLoader

Extracts text content from a static web page using requests + BeautifulSoup.

CSVLoader

Converts each row of a CSV file into a separate Document object.

I. Interview Questions

- Q1.** What problem does RAG solve that a plain LLM cannot?
- Q2.** What are the two components of every Document object in LangChain, and what does each contain?
- Q3.** Why do document loaders always return a list of Document objects instead of a single object?
- Q4.** How does PyPDFLoader decide how many Document objects to create from a single PDF?
- Q5.** What is the purpose of the glob parameter in DirectoryLoader, and how does it work?
- Q6.** Explain the difference between load() and lazy_load(). When would you choose one over the other?
- Q7.** What two libraries does WebBaseLoader rely on internally, and what does each one do?
- Q8.** Why does WebBaseLoader struggle with JavaScript-heavy web pages, and what alternative loader addresses this?
- Q9.** How does CSVLoader map rows in a CSV file to Document objects?
- Q10.** What steps would you take to build a custom document loader for a data source LangChain does not support?

F. Further Reading

- LangChain Document Loaders documentation (`langchain_community.document_loaders`)
- LangChain Document Loaders integrations catalog (full list of available loaders by category)
- pypdf library documentation (used internally by PyPDFLoader)
- BeautifulSoup documentation (used internally by WebBaseLoader for HTML parsing)
- LangChain BaseLoader API reference (for building custom document loaders)